Data Structures and Algorithms
Class Test 2, May 7, 2026

Instructions:

- Please fill in your student number (and if possible name) correctly.
- Answer all questions by correctly filling the appropriate answer in the answer sheet.
- Please return only the answer sheet, and discard the question paper later.

Q1.

You are given a reference to a node B in a circular singly-linked list. B is neither the first nor the last node. You wish to delete node B. Which of the following strategies allows you to effectively "remove" the data of node B from the list in $O(1)$ time? The next field of the last node in a circular singly-linked list stores the reference of the first node in the list.

(a) Travere the list to find the node A such that $A.next = B$, then set $A.next = B.next$.

(b) Copy the data from $B.next$ into B, then set $B.next = B.next.next$.

(c) Set $B = B.next$.

(d) It is impossible to delete node B in $O(1)$ time without a reference to the preceding node.

Q2.

A sorting algorithm is considered *stable* if it preserves the relative order of records with equal keys. Which of the following statements regarding sorting stability is **false**?

(a) Merge Sort is naturally stable because of how the merge step is typically implemented.

(b) Insertion Sort is stable because swaps do not reorder equal elements.

(c) Heap Sort is a stable sorting algorithm because it uses a structured binary tree.

(d) Any unstable sort can be made stable by adding the original index of the element as a secondary key.

Q3.

You are using a hash table of size 7 with the hash function $h(k) = k \mod 7$. You use linear probing to resolve collisions. After inserting the keys 15, 22, and 8 in that order, what is the index of the key 8?

(a) 1

(b) 2

(c) 3

(d) 4

Q4.

A specific binary tree has the following traversal sequences:

- **In-order:** D, B, E, A, F, C
- **Pre-order:** A, B, D, E, C, F

What is the **Post-order** traversal of this tree?

(a) D, E, B, F, C, A

(b) D, B, E, F, C, A

(c) E, D, B, F, C, A

(d) A, C, F, B, E, D

Q5.

A binary tree (not necessarily a complete binary tree) is stored in an array (starting at index 1). For any node at index i, its left child is at index $2i$ and its right child is at index $2i + 1$. Given the array $[2, \mathit{empty}, 3, \mathit{empty}, \mathit{empty}, 15, 20, 18, 11]$, which index violates the binary tree property first? `empty` indicates a node that does not exist.

(a) Index 3

(b) Index 6

(c) Index 8

(d) No violation exists.

Q6.

Consider a dynamic array that starts with a capacity of 1. When the array is full, its capacity is doubled. If an insertion costs 1 unit plus the cost of copying elements during a resize, what is the **amortized** cost of n insertions?

(a) $O(1)$

(b) $O(\log n)$

(c) $O(n)$

(d) $O(n \log n)$

Q7.

You are given two sorted arrays A is of size m and B is of size n. You want to find the k-th smallest element in the combined set $A \cup B$. What is the best possible worst-case time complexity to achieve this?

(a) $O(1)$

(b) $O(\log(\min(m, n)))$

(c) $O(\log(m + n))$

(d) $O(\sqrt{n})$

Q8.

In a hash table using *Double Hashing*, the primary hash function is $h_1(k) = k \mod 11$ and the secondary hash function is $h_2(k) = k \mod 7$. The second hash function is used if the index from the first hash function is already occupied. If a collision occurs at $h_1(58)$, the value of $h_2(58)$ is used as the next index to check. What is the index of the **first** probe (the first alternative slot checked) when there is a collision for $h_1(58)$?

(a) 3

(b) 2

(c) 6

(d) 8

Q9. Consider a min-heap containing n elements stored in an array. Which of the following statements is true?

(a) The smallest element in the heap can be found in $O(\log n)$ time.

(b) The height of a complete binary tree with n nodes is $O(\log n)$, ensuring that the worst-case time for a single insertion is $O(\log n)$.

(c) Deleting the minimum element from the heap and then restoring the heap property takes $O(n)$ time in the worst case.

(d) Any arbitrary unsorted array of size n can be treated as a valid min-heap without reordering the elements.

Q10.

You are given a reference `curr` to a node in a doubly-linked list. Each node has `next` and `prev` references. You want to swap `curr` with its successor `curr.next` (You want to swap their positions). Assuming `curr.next` is not null, which of the following code snippets correctly implements this swap? Assume that `Node` is the class that is used for creating a new node. Assume that there are at least 10 nodes to the left and right of `curr`.

(a)
```
curr.prev.next = curr.next
curr.next.prev = curr.prev
curr.next.next.prev = curr
curr.next = curr.next.next
curr.prev = curr.prev.next
curr.prev.next = curr
```

(b)
```
Node succ = curr.next
curr.prev.next = succ
succ.prev = curr.prev
curr.next = succ.next
succ.next.prev = curr
succ.next = curr
curr.prev = succ
```

(c)
```
curr.next = curr.next.next
curr.next.prev = curr
curr.prev.next = curr.next
curr.next.prev = curr.prev
curr.next.next = curr
```

(d)
```
Node temp = curr.prev
curr.prev = curr.next
curr.next = temp
curr.prev.next = curr
curr.next.prev = curr
```

End of Paper